# Caddy Media Gallery

## Operator Documentation

A Caddy v2 module for serving image and video galleries with a unified lightbox, configurable sort/pagination, and a templatable UI.

Generated June 24, 2026

<!– Comment moved here so it doesn't render as LaTeX text in the output. The TOC command below generates the actual TOC. –>

# Contents

# Configuration

## Caddyfile directive

```
handle_path /images/* {
    root * /var/www/html/images
    media_gallery
    file_server
}
```

The `media_gallery` directive accepts one inline option:

| Subdirective | Value | Default | Purpose |
|---|---|---|---|
| `template` | file name, relative to the templates dir | `gallery.tmpl` | Pick which template file to render. Path-traversal protected: no `..`, no absolute paths — the templates dir is a chroot. |
| `no_thumbs` | `true` / `false` (no-arg = `true`) | `false` (thumbs on) | Skip on-the-fly WebP thumbnail generation for images. Tile `<img src>` points to the original file instead of `~/_thumbs/<name>.webp`. Thumb requests fall through to the next handler. Useful for small galleries where you don't want a thumb cache. See `no_thumbs` walkthrough below. |

| Subdirective | Value | Default | Purpose |
|---|---|---|---|
| `no_video_thumbs` | `true` / `false` (no-arg = `true`) | `false` (video thumbs on, if ffmpeg available) | Skip on-the-fly WebP thumbnail generation for videos (extracted from the first frame via ffmpeg). When `true`, videos still display in the gallery (with the placeholder gradient + play button on each tile) but no per-frame thumbnail is generated. When `false` (default), video thumbs ARE generated IF ffmpeg is available on the host. If ffmpeg is missing, video thumbs fall back to the placeholder regardless of this setting (we can't decode a frame without a tool that can). Use `no_video_thumbs` to skip the ffmpeg invocation even when it's available (e.g., on hosts where you don't want the CPU cost of frame extraction). See "Video thumbnails (ffmpeg)" below. |

| Subdirective | Value | Default | Purpose |
|---|---|---|---|
| `page_size` | integer >= 1 | `50` | How many image entries to show per page. Must be a positive integer; `page_size 0` is rejected (use no directive, or set the explicit value you want). The pagination nav only renders when total pages > 1, so a 30-media gallery at the default 50 shows all 30 on a single page with no nav. |

Example with a themed subdir:

```
handle_path /images/* {
    root * /var/www/html/images
    media_gallery {
        template themes/dark/gallery.tmpl
    }
    file_server
}
```

This loads `$GALLERY_TEMPLATES_DIR/themes/dark/gallery.tmpl` (or falls back to the bundled template if the file doesn't exist on disk). The path is validated at Provision — an invalid name (e.g. `../etc/passwd`) fails Caddy startup, not at first request.

Example: skip thumbnail generation

```
handle_path /images/* {
    root * /var/www/html/images
    media_gallery {
        no_thumbs
    }
    file_server
}
```

With `no_thumbs`: - Each tile's `<img src>` is the original image file (`./photo.jpg`), not `~/_thumbs/photo.webp` - No thumb generation, no cache, no CPU cost on first request - The browser downloads the full image per tile (bigger page payload, slower on dirs of large photos)

Use `no_thumbs false` to turn it back on (the default is off, so the directive is opt-in).

Example: disable video thumbnails

Video thumbnails (per the table above) require ffmpeg. If ffmpeg isn't available, the gallery falls back to the placeholder gradient + play button automatically — you don't need to do

anything. If ffmpeg IS available and you want to skip frame extraction (e.g., on a low-CPU host or for very large videos), use `no_video_thumbs`:

```
media_gallery {
    no_video_thumbs
}
```

With this, video tiles show the placeholder gradient + play button (no `<img>` for the frame). The same `no_video_thumbs false` form re-enables frame extraction.

The video thumb generation uses `ffmpeg -vframes 1` to extract the first frame, scaled to fit the configured `thumb_width` × `thumb_height` (defaults 320×320). The output is a WebP, written to the same cache dir as image thumbs (`/var/cache/caddy-gallery` by default, override via `GALLERY_THUMB_CACHE_DIR`). Same caching rules as image thumbs (regenerate only when the source video's mtime is newer than the cache file).

If the operator has ffmpeg installed at a non-standard path (not in `$PATH`), set the `FFMPEG_PATH` env var to the absolute path of the ffmpeg binary. This is checked first (Phase 67); if unset or the file isn't executable, the code falls back to `exec.LookPath("ffmpeg")` which scans `$PATH`.

```
FFMPEG_PATH=/opt/ffmpeg-7/bin/ffmpeg caddy run
```

The `FFMPEG_PATH` value is validated at Provision time: it must point to an existing regular file with at least one executable bit set. Bad values (non-existent path, directory, non-executable file) are silently ignored and the code falls back to `$PATH` lookup. This avoids a confusing "exec: not found" error at request time when the env var is mistyped.

All standard install paths (`/usr/bin/ffmpeg`, `/usr/local/bin/ffmpeg`, etc.) are picked up automatically via the `$PATH` fallback. Best for: small galleries (< 100 images) where you don't want a thumb cache and the originals aren't huge. Not recommended for large galleries — the page payload goes from ~30 KB (with thumbs) to ~5 MB average (full images) for a 1,000-image dir.

Example: change the page size

```
handle_path /images/* {
    root * /var/www/html/images
    media_gallery {
        page_size 100
    }
    file_server
}
```

This shows 100 image entries per page instead of the default 50. Tradeoffs: larger pages mean fewer HTTP requests, but each request returns a bigger HTML payload (and the server uses more memory per render). The pagination nav at the bottom of the page only renders when total pages > 1, so if your gallery has 30 images and you set `page_size 100`, you get all 30 on one page with no nav. URL query override: append `?page=2` to the gallery URL to jump to a specific page. `?page_size=N` is NOT a query param — page size is set in the Caddyfile only (per-request override would let the user request arbitrarily large pages and could DOS the server).

All other configuration (the `root *` for the image directory, the `handle` / `handle_path` for the route, the auth wrapper) is via the surrounding Caddyfile block, not the `media_gallery` direc-

tive. The module reads the on-disk directory from Caddy's per-request `root` variable, or from the `Root` JSON field if set explicitly.

## JSON config (advanced)

Caddy supports two configuration formats: the Caddyfile (text, what most examples on this page use) and JSON (the native config format Caddy uses internally). Every Caddyfile gets converted to JSON before being applied — but you can also write JSON directly, which is useful for:

- Programmatic / templated config (Kubernetes, Terraform, Ansible) — JSON can be generated from variables
- Many Caddy instances — JSON is diffable, lintable, and can be validated in CI pipelines
- Dynamic reload — `caddy reload` accepts JSON via the admin API (`curl -X POST http://localhost:2019/load`)
- Sharing snippets — unambiguous quoting (no special-character rules like Caddyfile)

### Minimum JSON config

The minimum handler block (only `handler` is required):

```
{
  "handler": "media_gallery",
  "root": "/var/www/html/images"
}
```

The `Root` field is optional — the module falls back to the per-request `root` set by the surrounding `handle` / `handle_path` block. Set it explicitly only if you need to override the request-time root.

### Full JSON config (all fields)

Here's a complete JSON config showing every configurable field of the `media_gallery` handler, with realistic values:

```
{
  "handler": "media_gallery",
  "root": "/var/www/html/images",
  "sort": "name",
  "template": "gallery.tmpl",
  "no_thumbs": false,
  "no_video_thumbs": false,
  "page_size": 50,
  "thumb_width": 320,
  "thumb_height": 320,
  "thumb_format": "webp",
  "thumb_ttl": 1440,
  "cache_scan": 1
}
```

All fields are optional except `handler` (always required). Defaults match the Caddyfile defaults — if you omit a field, the same default value applies.

Caddyfile [] JSON field mapping

| Caddyfile directive | JSON field | Type | Default |
|---|---|---|---|
| `root` (per-handle) | `"root"` | string | (per-request) |
| `sort <name\|mtime\|type\|size>` | `"sort"` | string | `"mtime"` |
| `template <name>` | `"template"` | string | `"gallery.tmpl"` |
| `no_thumbs` | `"no_thumbs"` | bool | `false` |
| `no_video_thumbs` | `"no_video_thumbs"` | bool | `false` |
| `page_size <N>` | `"page_size"` | int | `50` |
| `thumb_width <N>` | `"thumb_width"` | int | `320` |
| `thumb_height <N>` | `"thumb_height"` | int | `320` |
| `thumb_format <webp\|jpeg\|png>` | `"thumb_format"` | string | `"webp"` |
| `thumb_ttl_minutes <N>` | `"thumb_ttl"` | int | `1440` (24h) |
| `cache_scan_minutes <N>` | `"cache_scan"` | int | `1` |

Heads up on the JSON naming: the JSON field names use the `json:"name"` struct tags — for `CacheScanMinutes` the tag is `cache_scan` (not `cache_scan_minutes`), and for `ThumbTTLMinutes` it's `thumb_ttl` (not `thumb_ttl_minutes`). This is intentional: the Go struct tags are short, and the Caddyfile subdirectives keep the verbose names. The mapping table above is authoritative for what JSON field names to use — the Go field names are an implementation detail.

The mapping for the other fields is mechanical: every Caddyfile subdirective has a matching JSON field with the same name (snake_case throughout). The Go struct field names are `Root`, `Sort`, etc. (PascalCase), but the JSON tags normalize them to snake_case via the `json:"name, omitempty"` struct tags.

### Validation

You can validate a JSON config without starting Caddy:

```
caddy validate --config /etc/caddy/caddy.json
```

Output: `Valid configuration` (or a JSON parse error pointing to the problem). The `caddy validate` command works for both JSON and Caddyfile inputs — for Caddyfile, just point it at the file directly.

### Dynamic reload

Push a JSON config to a running Caddy via the admin API:

```
curl -X POST http://localhost:2019/load \
  -H "Content-Type: application/json" \
  --data-binary @new-config.json
```

Returns `200 OK` on success, or a JSON error response on failure. This is what `caddy reload` does internally — but you can also script reloads (e.g., update the gallery config when the disk layout changes) by POSTing to this endpoint.

When to use JSON vs Caddyfile

For single-host, edit-by-hand use, Caddyfile is the right tool: simpler for humans, more forgiving of whitespace and comments, easier to read for someone not familiar with the schema. This is what most of this document shows.

For automated / multi-instance / programmatic use, JSON is the right tool: it's diffable, lintable, validatable, and can be generated from templates. This is what Caddy uses internally, and what most CI/CD pipelines produce.

You can also convert between them: `caddy adapt --config Caddyfile` produces JSON on stdout, and `caddy adapt --config caddy.json --adapter caddyfile` goes the other way (rare, since the Caddyfile syntax is less expressive).

## Environment variables

The plugin reads one environment variable:

`GALLERY_TEMPLATES_DIR`

Where it's read in code: `render.go`, in two functions — `loadTemplate()` (at request time, to find the on-disk override) and `writeBundledTemplates()` (at Caddy startup, to write the bundled templates so operators can see them). Both do the same thing:

```
dir := os.Getenv("GALLERY_TEMPLATES_DIR")
if dir == "" {
    dir = "/etc/caddy/gallery-templates"
}
```

Default: `/etc/caddy/gallery-templates`. Created on first Caddy startup by `writeBundledTemplates()` with mode 0*755*, owned by whatever user the Caddy systemd service runs as. If the template file already exists, it's left alone (operator overrides are preserved across restarts).

The directory it points to: the absolute path you set. The plugin does not create parent directories beyond the templates dir itself — it just `MkdirAll`s the final directory. So if you point it at `/srv/gallery-templates`, that path needs to be writable by the Caddy process.

How to set it for the live Caddy (the canonical way — systemd starts the process with a clean environment, so your shell's env doesn't reach the service):

```
# One-off (persists in the manager's env, inherited by all units)
sudo systemctl set-environment GALLERY_TEMPLATES_DIR=/etc/caddy/gallery-templates
sudo systemctl restart caddy

# Or persistently in the caddy service unit
sudo systemctl edit caddy
# Add:
#   [Service]
#   Environment="GALLERY_TEMPLATES_DIR=/etc/caddy/gallery-templates"
# Or use EnvironmentFile= pointing at a file with the line
sudo systemctl daemon-reload
sudo systemctl restart caddy
```

How to set it for dev (`go test`, `xcaddy build`, or running a custom Caddy from your shell):

```
export GALLERY_TEMPLATES_DIR=/path
```

```
2 # then run your commands
```

Note: the test suite sets `GALLERY_TEMPLATES_DIR` to a temp dir via `TestMain`, so `go test` is isolated from your shell's value. The module always reads the env at request time, so changes to the env var take effect on the next Caddy restart — no rebuild needed.

What happens if the dir is unwritable or doesn't exist: `writeBundledTemplates()` logs a warning to stderr and continues. The bundled templates still serve fine (the on-disk file is a convenience, not a requirement). The next request falls back to the bundled constant.

There are no other env vars. Sort order, pagination, page size, and the thumb cache directory are all configurable in code (constants in the source) rather than via env vars.

## In-code constants (not configurable)

These are baked into the source. They can be changed by editing the code, rebuilding, and restarting Caddy — but they're not exposed as Caddyfile directives or env vars. Listed here so you know what the defaults are and where to find them.

| Constant | Value | Where |
|---|---|---|
| Thumbnail size | 320px | `thumbnails.go` |
| Thumb `Cache-Control` max-age | 86400 (24h) | `thumbnails.go` |
| Thumb format | WebP (lossless, via `nativewebp`) | `thumbnails.go` |
| Scan cache TTL | 1 minute | `gallery.go` (`NewScanCache(time.Minute)`) |
| Thumb width (px) | 320 | `thumbnails.go` (the resize) |

Note: "Thumbnail size" and "Thumb width (px)" both reference the same constant (320) — the maximum dimension of the generated thumbnail. If you wanted them to be different values (say, max-dim 320 and width 280 for a non-square crop), let me know and I'll split them into two separate constants.

Why these aren't configurable yet: most operators never need to change them. The thumb size and Cache-Control max-age are sensible defaults that work for the vast majority of galleries. The scan cache TTL is short enough (1 minute) that new files appear quickly without manual cache busting, and long enough that hot directories don't re-scan on every request. If you find yourself wanting to change one of these, that's a signal that the constant should probably be promoted to a config option — file an issue and we can discuss.

## What `media_gallery` does NOT do

- It does not generate thumbnails at build time — thumbnails are generated on-the-fly on first request and cached in `/var/cache/caddy-gallery/`. See the wiki page for cache details.
- It does not handle uploads or modifications. It's read-only.
- It does not support nested directory pagination — when you click into a subdirectory, that subdirectory has its own gallery with its own pagination, but the parent dir's image list isn't merged in.
- It does not redirect from `/foo` to `/foo/`. If you request `/images/generated` (no trailing slash) and it's a directory, the module returns a 301 with a relative `Location: generated/` header (so the browser resolves it relative to the current URL). This is a workaround for Caddy's

handle_path rewriting both r.URL.Path and r.RequestURI, which makes absolute reconstruction impossible inside the handler.

# caddy_media_gallery — Complete configuration reference

A single-page reference for every configuration knob the media_gallery Caddy handler exposes. Use this as the "what can I configure" reference; for how to use individual features, see the per-topic docs linked below.

For per-topic deep dives: - configuration.md — Caddyfile directive details + env vars - templates.md — the template file + variables - sort-and-pagination.md — the URL query API

---

## 1. Caddyfile subdirectives (the primary way)

Inside an media_gallery { … } block:

| Subdirective | Value | Default | Purpose |
|---|---|---|---|
| sort | mtime / name (also accepts date as an alias for mtime — see Sort & Pagination to Aliases) | mtime (newest first) | Default sort field. Overridable per-request via ?sort=. |
| template | file name, relative to the templates dir | gallery.tmpl | Which template file to render. Path-traversal protected. |
| no_thumbs | no-arg = true / explicit false = false | false (thumbs on) | Skip thumbnail generation. Tile <img> points to the original file. |
| page_size | integer >= 1 | 50 | Image entries per page. Nav only renders when total pages > 1. |
| thumb_width | integer >= 1 | 320 | Max width in pixels. Source is fit-within-bounds. |
| thumb_height | integer >= 1 | 320 | Max height in pixels. Source is fit-within-bounds. |
| thumb_format | jpeg / jpg / png / webp | webp (lossless) | Output format. jpeg quality 75, png lossless, webp lossless. |
| cache_scan | integer >= 1 | 1 | Scan cache TTL in minutes. |
| thumb_ttl | integer >= 1 | 1440 | HTTP Cache-Control: max-age for thumbs, in minutes (= 24h default). |

Full Caddyfile example (every directive set):

```
handle_path /images/* {
    root * /var/www/html/images
    media_gallery {
        sort name
        template themes/dark/gallery.tmpl
        no_thumbs false
        page_size 100
        thumb_width 480
        thumb_height 320
        thumb_format jpeg
        cache_scan 5
        thumb_ttl 60
    }
    file_server
}
```

---

## 2. JSON config fields (programmatic)

Same fields, json tags. Use `caddy adapt` or any JSON config producer to set them:

```
{
  "root": "/var/www/html/images",
  "sort": "name",
  "template": "themes/dark/gallery.tmpl",
  "no_thumbs": false,
  "page_size": 100,
  "thumb_width": 480,
  "thumb_height": 320,
  "thumb_format": "jpeg",
  "cache_scan": 5,
  "thumb_ttl": 60
}
```

`cache` is runtime-only (excluded from JSON via `json:"-"`).

---

## 3. Per-request URL query parameters (no config needed)

| Param | Values | Default | Effect |
|---|---|---|---|
| `sort` | `mtime` / `name` / `type` / `size` (also accepts `date` as an alias for `mtime` — see Sort & Pagination to Aliases) | inherits from Caddyfile | Sort field |
| `order` | `asc` / `desc` | depends on `sort` | Sort direction |
| `page` | integer >= 1 | 1 | Which page (only meaningful when `page_size` causes pagination) |

Deliberately NOT query-overridable: - `?page_size=N` — would let users request arbitrarily large pages and could DOS the server - `?thumb_format=N` — would let users force the server to recompute thumbs in any format - `?thumb_width=N` — same

The page size, format, and thumb dimensions are set in the Caddyfile only.

---

## 4. Environment variables

| Var | Default | Purpose |
|---|---|---|
| `GALLERY_TEMPLATES_DIR` | `/etc/caddy/gallery-templates` | Where the module looks for (and writes) the on-disk template override. The only user-facing env var. |

A second env var exists in the code for testing only: `GALLERY_THUMB_CACHE_DIR` (default `/var/cache/caddy-gallery`) — not documented as a user-facing knob.

See configuration.md to Environment variables for the full setup story (systemd unit, dev workflow, failure mode).

---

## 5. Caddyfile handler-level wiring (external, not `media_gallery`-specific)

These live in the surrounding Caddyfile, not inside the `media_gallery { ... }` block:

| Caddyfile construct | Purpose | Example |
|---|---|---|
| `handle_path` (or `route`) | Strips URL prefix and routes to `media_gallery` | `handle_path /images/* { ... }` |
| `root *` | Per-request "root" var, read by `media_gallery` to find the directory to scan | `root * /var/www/html/images` |
| `file_server` | Fallthrough for direct file requests and 404s (must come AFTER `media_gallery`) | `file_server` |
| Auth (Authelia, basic_auth, etc.) | Wraps the route in auth | `(auth)` snippet (or `basicauth { ... }` block) |

---

## 6. External: the Caddyfile as a whole

The live site typically looks like:

```
hermes.synapticloop.com {
    import auth
    route /images/* {
        root * /var/www/html/images
        media_gallery
        file_server
    }
}
```

The `(auth)` snippet is defined elsewhere in the Caddyfile and wraps every route in Authelia `forward_auth` at `127.0.0.1:9091`.

---

## 7. Behavior modifiers (not configurable, hardcoded)

These are baked into the source. They could be promoted to config options if a use case comes up, but currently aren't:

| Constant | Value | Where |
|---|---|---|
| Thumb JPEG quality (when `thumb_format jpeg`) | 75 | `thumbnails.go` |
| Video thumbs | not generated (videos show play-button overlay in the template) | `thumbnails.go` (source extensions are image-only) |
| Lightbox | page-scoped (50 images on the current page, not all 1,197) | `render.go` (lightbox JS in the template) |
| Other-files strip on a subdir page | "Other files" rendered as horizontal chips above the image grid | `render.go` (splitFiles) |
| Directories strip on every page | Always rendered, alphabetical, ignores sort | `render.go` (splitFiles + dirs sort) |

---

## 8. Quick reference — what's where

If you're wondering "where is this knob?":

| Want to configure… | Use |
|---|---|
| Sort field (default + per-request) | `sort` directive + `?sort=` query |
| Pagination | `page_size` directive + `?page=` query |
| Thumbnail on/off | `no_thumbs` directive |
| Thumbnail size | `thumb_width` / `thumb_height` directives |
| Thumbnail format (jpeg/png/webp) | `thumb_format` directive |
| Thumbnail browser cache TTL | `thumb_ttl` directive |
| Directory scan cache TTL | `cache_scan` directive |
| Template file (theme) | `template` directive (e.g. `themes/dark/gallery.tmpl`) |
| Templates directory (where to find templates) | `GALLERY_TEMPLATES_DIR` env var |
| Thumbnails cache directory | `GALLERY_THUMB_CACHE_DIR` env var (testing only) |
| Auth (who can view) | Caddyfile `basicauth` or `(auth)` snippet (external) |
| Route (URL prefix) | Caddyfile `handle_path` or `route` (external) |
| Image directory | Caddyfile `root *` (external) |

# Templates

The gallery is rendered by an `html/template` (Go's standard template engine). The template is bundled in the binary as a Go string constant (`galleryTemplate` in `render.go`), and on each Caddy startup the module also writes it to disk at `/etc/caddy/gallery-templates/gallery.tmpl` (or `$GALLERY_TEMPLATES_DIR` if set). The on-disk file exists so operators can read the template without opening the binary, and so they can override individual templates by editing the file in place.

## How the template loading works

```
loadTemplate(name string) reads $GALLERY_TEMPLATES_DIR
    │
    ├── name = gallery.Template (from Caddyfile `template` directive)
    │   default "gallery.tmpl" if the directive is absent
    │
    ├── sanitizeTemplateName(name):
    │   reject absolute paths, reject ".." (any traversal)
    │
    ├── /etc/caddy/gallery-templates/<clean name> exists?
    │     ├── YES to parse that file directly (CSS+JS are inside it)
    │     │         (the on-disk file is the active template)
    │     └── NO  to use the bundled galleryTemplate constant
    │
    └── return *template.Template ready to RenderPage
```

The `template` Caddyfile directive is the operator-facing knob that picks the template file. See `docs/configuration.md` for the directive syntax and the path-traversal protection details.

Note on `no_thumbs`: the template is unaffected by the `no_thumbs` Caddyfile directive. The template always uses `{{.ThumbURL}}` for the tile `<img src>`; the `no_thumbs` flag changes the *value* of that field (to the original file URL when true, to the thumb URL when false), not the field name or its usage. So a template that works with thumbs also works with no_thumbs (and vice versa), with no template changes needed.

The template is a single self-contained file. The HTML, CSS (inside `<style>`), and JS (inside `<script>`) all live in one Go string constant (`galleryTemplate` in `render.go`) and one on-disk file (`/etc/caddy/gallery-templates/gallery.tmpl`). There is no sub-template loading — the previous Phase 16 design that split them into 3 files (`gallery.tmpl` + `style.css` + `lightbox.js`) was collapsed in Phase 17 for easier editing.

The bundled constant is the source of truth at build time. The on-disk file is an *override* layer; if you delete it, the module falls back to the bundled version automatically on the next request.

## What's in the single template file

One file, `gallery.tmpl`, ~16.7 KB / 574 lines, containing:

| Section | What's in it | Approx lines |
|---|---|---|
| `<!DOCTYPE>` … `<style>` | Document head, including the full CSS inlined as a `<style>` block | ~10 + 365 (CSS) |

| Section | What's in it | Approx lines |
|---|---|---|
| `<body>`, `<main>`, `<header>` | Title, counts, sort indicator, sort bar | ~50 |
| Directories section | `{{if .Directories}}` … `{{end}}` — the dirs chip strip | ~7 |
| Other files section | `{{if .OtherFiles}}` … `{{end}}` — the others chip strip | ~7 |
| Images section | Sort info, paginated grid, per-tile HTML | ~80 |
| Pagination | `{{if gt .TotalPages 1}}` … `{{end}}` — prev/next links | ~15 |
| `<script>` … `</script>` | The full JS inlined (lightbox, open-in-new-tab, sort indicator) | ~100 (JS) |

The CSS rules and the HTML they apply to are interleaved top-to-bottom in the file, so when you scroll through the template you see the structure (HTML), the styling (CSS), and the behavior (JS) in document order. Easier to hand-edit than three separate files.

## Template variables

The template receives a `PageData` struct. All fields you can reference from `{{...}}` are listed below.

Top-level fields (on `PageData`)

| Field | Type | What it's for |
|---|---|---|
| `{{.Title}}` | `string` | The page title (rendered in `<title>` and the `<h1>`). Currently the directory name. |
| `{{.PathPrefix}}` | `string` | Prefix for relative links to files in the same directory. Usually `"./"`. |
| `{{.ThumbPrefix}}` | `string` | Prefix for thumbnail URLs. Usually `"./_thumbs/"`. |
| `{{.Directories}}` | `[]FileView` | Subdirectory entries. Always rendered in full (not paginated), case-insensitive alphabetical. |
| `{{.OtherFiles}}` | `[]FileView` | Non-media files (HTML, txt, etc.). Always rendered in full (not paginated). |
| `{{.Images}}` | `[]FileView` | The image + video tiles for the current page only. Paginated, sorted per the user's `?sort=&order=` choice. |
| `{{.Page}}` | `int` | Current page number (1-based). |
| `{{.PageSize}}` | `int` | Images per page (constant; default 50). |

| Field | Type | What it's for |
|---|---|---|
| `{{.TotalImages}}` | `int` | Total images in the directory (across all pages). |
| `{{.TotalPages}}` | `int` | Total page count. |
| `{{.HasPrev}}` | `bool` | True if `{{.Page}} > 1`. |
| `{{.HasNext}}` | `bool` | True if `{{.Page}} < {{.TotalPages}}`. |
| `{{.Sort}}` | `SortSpec` | The current sort. See below. |

`{{.Sort}}` (a `SortSpec` struct)

| Field | Type | What it's for |
|---|---|---|
| `{{.Sort.Field}}` | `string` | One of `"mtime"`, `"name"`, `"type"`, `"size"`. (The `?sort=` URL param. `"mtime"` is the default.) |
| `{{.Sort.Order}}` | `string` | `"asc"` or `"desc"`. (The `?order=` URL param.) |

Per-entry fields (on `FileView`)

When you `{{range .Images}}` (or `.Directories` / `.OtherFiles`), each iteration gives you a `FileView` with these fields:

| Field | Type | What it's for |
|---|---|---|
| `{{.Name}}` | `string` | The basename (`"photo.jpg"`, `"subdir"`, etc.). Truncated with ellipsis in the live template if too long for the tile. |
| `{{.Href}}` | `string` | Relative link to the file. Use as `<a href="{{.Href}}">`. |
| `{{.ThumbURL}}` | `string` | For images and videos, the relative thumbnail URL (e.g. `./_thumbs/photo.webp`). Empty string for non-media files — check `{{if .ThumbURL}}` before using. |
| `{{.IsDir}}` | `bool` | True for directories. |
| `{{.IsImage}}` | `bool` | True for image files. |
| `{{.IsVideo}}` | `bool` | True for video files. Videos go in the image grid with a play-button overlay (no `<img>` child); `{{.ThumbURL}}` is set but the live template doesn't render an `<img>` for them. |
| `{{.IsOther}}` | `bool` | True for non-media files (HTML, txt, etc.). |

| Field | Type | What it's for |
|---|---|---|
| `{{.Type}}` | `string` | Uppercase extension without the dot: `"JPG"`, `"DIR"`, `"MP4"`, `"HTML"`, etc. |
| `{{.Size}}` | `string` | Human-readable file size: `"234 KB"`, `"1.2 MB"`, etc. Empty string for directories. |
| `{{.Date}}` | `string` | ISO date `"2006-01-02"` (UTC-normalised). Empty string for directories. |

Template functions (the funcmap)

The template engine has a few helper functions registered:

| Func | Signature | What it does |
|---|---|---|
| `minus1` | `minus1 n int to int` | Returns `n - 1`. Used for prev-page link targets. |
| `plus1` | `plus1 n int to int` | Returns `n + 1`. Used for next-page link targets. |
| `sortLabel` | `sortLabel field string to string` | Maps a sort field code to its display label: `"name"to"Name"`, `"type"to"Type"`, `"mtime"to"Modified"`, `"size"to"Size"`, `"date"to"Date"`. Unknown fields fall back to the raw field name capitalised. Empty string to `"Modified"` (the default). |

## Editing the templates — the basics

1. Find the templates dir. On this host it's `/etc/caddy/gallery-templates/`. The file is `gallery.tmpl`.
2. Edit the file in place. The module re-reads it on every request, so changes take effect immediately — no Caddy restart needed.
3. Test in a browser. The next request will use the new template. If the template has a parse error, the module will return 500 with the parser error in the response body. Fix the syntax and try again.
4. Revert to the bundled version. If you want to go back to what the binary ships with, delete the on-disk file: `sudo rm /etc/caddy/gallery-templates/gallery.tmpl`. The next request falls back to the bundled constant.

## Walkthrough: change the theme to dark mode

Since the CSS is now inlined in the same file as the HTML, you edit the `<style>` block directly in `gallery.tmpl` — no second file to coordinate, no sub-template indirection.

1. Open `/etc/caddy/gallery-templates/gallery.tmpl` in your editor.
2. Find the `<style>` block. It's the big CSS block near the top of the file, just after `<title>…</title>`. Search for `html, body { background: #f3f6f7;` to find the body color.

3. Change the colors inline. The CSS is plain CSS — no preprocessing. Most of the theme colors are concentrated in the first 50-100 lines.
4. Save the file. The change takes effect on the next request (no Caddy restart needed — the module re-reads the on-disk template on every `loadTemplate` call).
5. Hard-reload in the browser to bypass the HTML cache.

A common set of swaps for dark mode (find/replace these in the `<style>` block):

| Light (default) | Dark variant |
|---|---|
| `html, body { background: #f3f6f7; }` | `html, body { background: #1a1a1a; color: #ddd; }` |
| `main { background: white; }` | `main { background: #222; }` |
| `header { border-bottom: 1px solid #e5e9ea; }` | `header { border-bottom: 1px solid #333; }` |
| `.chip { background: #f3f6f7; border: 1px solid #e5e9ea; }` | `.chip { background: #2a2a2a; border: 1px solid #3a3a3a; }` |
| `a { color: #006ed3; }` | `a { color: #4ea3ff; }` |

## Walkthrough: add a “Created” column to the image tiles

If you want to show the file’s created time alongside the modified date (the template currently shows `Date` which is the modified date):

1. Note: `FileView.Date` is the only date field exposed. The `FileInfo` struct in the scanner *does* have `ModTime` (in nanoseconds) but no `CreatedTime`. Adding a “Created” column would require a code change — extend `FileView` with a `Created string` field, populate it in `RenderPage` from `info.ModTime` (or from `os.Stat` birthtime via `syscall.Statx` on Linux), then reference it in the template as `{{.Created}}`.

   In other words: this is a code change, not a template-only change. The template is fully driven by the Go struct fields — anything you want to display has to be in the struct.

## Where the bundled templates are defined in source

If you want to read the source (or fork and customise):

| File | Constant | Lines (approx) |
|---|---|---|
| `render.go` | `galleryTemplate` | line 392, ~574 lines (HTML + inlined CSS + inlined JS) |

A single constant. CSS and JS are inlined inside `<style>` and `<script>` blocks respectively. To customise: edit the constant, rebuild the module (`./build.sh`), restart Caddy (`sudo systemctl restart caddy`). The on-disk template is written from the new constant on the next startup.

## Upgrading from a pre-inlining install (Phase 16 to Phase 17)

If your site was running the old 3-file template split (`gallery.tmpl` + `style.css` + `lightbox.js`) and you upgraded to the new inlining build, the on-disk files are in an inconsistent state:

- `style.css` and `lightbox.js` from the old build are now dead weight. `writeBundledTemplates` removes them automatically on the next Provision after upgrade. Safe to ignore.

- The on-disk `gallery.tmpl` from the old build still has the old `{{template "lightbox.js" .}}` references, which no longer work. The new `loadTemplate` will fail to parse this file and the gallery will 500.

One-time fix on upgrade:

```
sudo rm /etc/caddy/gallery-templates/gallery.tmpl
sudo systemctl restart caddy
```

The next Provision writes the new inlined template. After that, the on-disk file is the canonical inlined version, and any operator edits to it become live overrides.

## Upgrading the bundled template content

`writeBundledTemplates()` deliberately does NOT overwrite an existing `gallery.tmpl` on disk — that's the operator-override contract. When the bundled template's contents change (e.g. new CSS rule, new template branch, fixed layout bug), the on-disk file is NOT updated automatically.

To pick up the new bundled content:

```
# 1. Save any local customisations (if you have any)
diff /etc/caddy/gallery-templates/gallery.tmpl      /home/osmanj/projects/
    caddy_media_gallery/render.go
# 2. Delete the on-disk file
sudo rm /etc/caddy/gallery-templates/gallery.tmpl
# 3. Restart Caddy so the next Provision runs writeBundledTemplates
sudo systemctl restart caddy
# 4. (Optional) Re-apply your local customisations
```

This workflow is intentional: operators who have customised the template (e.g. themed dark mode) keep their changes across upgrades. Operators who haven't customised just get the "fresh bundled version" after the rm + restart.

Future enhancement (not v1): writeBundledTemplates could write a content hash into a sidecar file, and on Provision, if the sidecar hash doesn't match the bundled hash, overwrite the on-disk file. This would auto-update without breaking the operator-override contract (the operator could still delete the on-disk file to fall back to the bundled version, OR write a sidecar with a custom hash to pin to their version).

## Troubleshooting

Edit took effect but the page looks the same. Hard-reload (Cmd-Shift-R / Ctrl-F5). The browser may have cached the previous HTML.

Edit took effect but the page is a 500. Your template has a parse error. The response body will contain the Go `html/template` parser error. Common causes: - Unclosed `{{if}}` / `{{range}}` / `{{with}}` blocks - `{{end}}` mismatch (most common — Go templates require `{{end}}` to close every block) - Calling a method that doesn't exist on the data type (e.g. `{{.Foo.Bar}}` when `Foo` is a string) - An unescaped backtick inside a Go template comparison (backticks terminate the Go raw string the constant is in)

Edit took effect but layout is broken. You removed or re-ordered a structural element. The CSS selectors and the JS `querySelector` calls expect a specific DOM shape — search for the class name in the template to see what's expected.

Want to test a new template before deploying it. Stage the edit in a new file at, say, `/etc/caddy/gallery-templates/gallery.tmpl.staging`, then copy it over the live file once you're happy. Or `curl` the gallery to see the live HTML and check it with a browser inspector before saving.

Got a 500 immediately after upgrading. You have the pre-inlining on-disk `gallery.tmpl` from a previous build. See the "Upgrading from a pre-inlining install" section above.

## Dark mode + theme toggle

The template supports three themes:

1. Auto (default) — follows the visitor's OS preference via the `prefers-color-scheme` CSS media query. macOS, Windows, iOS, and Android all expose this preference; if the visitor's OS is in dark mode, the gallery renders in dark mode automatically.
2. Light — forces light mode regardless of OS preference.
3. Dark — forces dark mode regardless of OS preference.

The choice is presented as a small toggle button group in the page header. Three buttons: a gear icon (Auto), a sun icon (Light), a moon icon (Dark). The current state is marked with `aria-pressed="true"` and a slightly inset background.

Position (Phase 50/51): the toggle was originally at the top-LEFT of the header (just to the left of the page title). As of Phase 50, it sits at the top-RIGHT, where the old sort-order indicator used to be. The sort indicator itself was removed in Phase 50 (the sort UI still exists as the sort-bar below the header — the active sort field is still visible there).

Mobile layout (Phase 50/51): on screens []600px wide, the header stacks vertically via `@media (max-width: 600px) { .header-top { flex-direction: column; }`. The visual order on mobile is:

1. Theme toggle (top, right-aligned via `align-self: flex-end`)
2. h1 + meta line (below, full width)

This is achieved with CSS `order` on the flex children, not by reordering the HTML source (the source order stays h1 [] meta [] toggle, which is better for screen readers):

```
1  .header-main { order: 2; }
2  .theme-toggle { order: 1; align-self: flex-end; }
```

The toggle's row order (`order: 1`) puts it above the header content (`order: 2`) on mobile; the `align-self: flex-end` keeps it right-aligned. On desktop (the default), neither `order` nor `flex-direction` applies, so the layout reverts to the normal horizontal flex with the toggle at the right (via the `justify-content: space-between` on `.header-top`).

The choice persists across visits via `localStorage` under the key `gallery-theme`. Values stored: `auto` | `light` | `dark`. `auto` is the default (no attribute set on `<html>`); `light` and `dark` set the `data-theme` attribute on `<html>`.

### No flash of wrong theme

There's a tiny inline `<script>` in the `<head>` that reads `localStorage` and applies the `data-theme` attribute to `<html>` BEFORE the body paints. This runs synchronously during HTML parsing, so the CSS already sees the right theme when the body renders. No flash of light theme when the visitor has chosen dark.

How dark mode is implemented

All colors are defined as CSS custom properties (CSS variables) in the `:root` selector at the top of the template's `<style>` block. There are ~16 tokens, divided into semantic groups:

| Group | Tokens |
|---|---|
| Backgrounds | `--bg` (page), `--bg-card` (cards), `--bg-chip` (chips), `--bg-hover` (chip hover), `--bg-active` (active chip) |
| Text | `--fg` (primary), `--fg-muted` (secondary), `--fg-faint` (tertiary), `--fg-disabled` (disabled) |
| Borders & shadows | `--border`, `--border-strong`, `--shadow`, `--shadow-strong` |
| Accents | `--accent` (links + borders), `--accent-hover`, `--accent-bg` (button fills — separate from `--accent` so the dark-mode button can be a muted darker blue without dimming link text) |

The dark mode override is just a second block of token assignments that applies when:

- `@media (prefers-color-scheme: dark) { :root:not([data-theme="light"]) { ... } }` — the OS is in dark mode AND the user hasn't explicitly picked light. The `:not([data-theme="light"])` selector is what makes the "Auto" state work: when the OS is dark but the user picked Light, the dark tokens don't apply.
- `[data-theme="dark"] { ... }` — manual dark mode regardless of OS preference. Triggered by clicking the moon icon; choice persists in localStorage.

Why `--accent` and `--accent-bg` are separate tokens: the accent color serves two visual roles — text/borders (good as a bright color on a dark bg, e.g. `#4dabff`) and button fills (looks glaring as a bright color, should be muted, e.g. `#3b6fb6`). Splitting them lets each role have its own dark-mode value.

Dark mode refinements (Phase 40–42)

The first dark-mode pass had two issues:

- Chip bg too light (Phase 40): the chips (directories, other files, sort buttons, page buttons) had a `--bg-chip` value (`#1d1d1d` in dark mode) that was visibly lighter than the page bg (`#1a1a1a`), making the chips stand out as bright blobs. Fixed by setting `--bg-chip: #1a1a1a` (same as `--bg`) in dark mode, so chips blend in with the page; only the border + text show. This mirrors the light-mode behavior (where `--bg-chip` already equals `--bg`).
- Hardcoded colors in 11 CSS rules (Phase 41): a regex-based refactor (the original dark-mode implementation) caught 18 rules but missed 11 more that used hardcoded light-mode hex values. After a comprehensive sweep, all CSS rules now use `var()` token references. The only `#hex` colors left in the on-disk template are: the token definitions themselves (light + dark overrides), the video tile placeholder gradient (theme-independent), and the play button colors (white-on-dark, theme-independent).

Customizing colors

To override a color, edit the on-disk template at `/etc/caddy/gallery-templates/gallery.tmpl` and change the token value in the `:root` block. For example, to make the accent color green instead of blue:

```
:root {
  --accent: #2e8b57;       /* was #006ed3 */
  --accent-hover: #3aa86a; /* was #0095e4 */
  --accent-bg: #2e8b57;    /* was #006ed3 — used by active sort/page buttons */
}
```

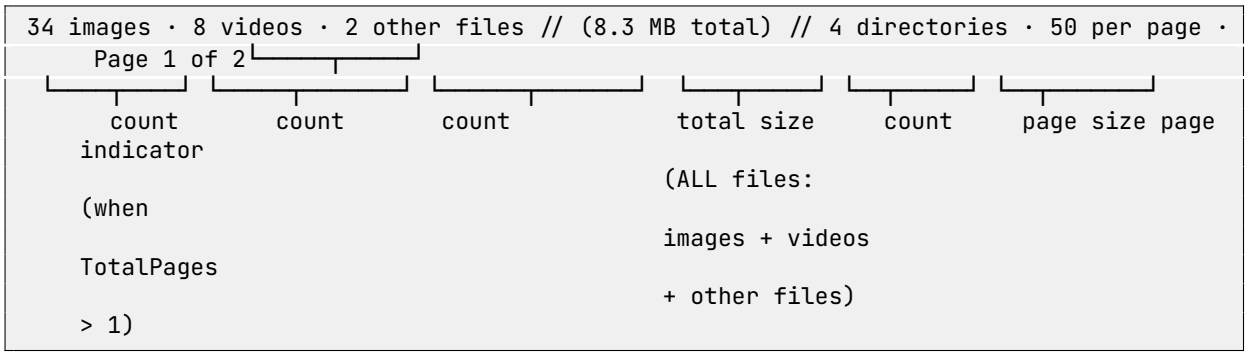
To add a new dark-mode-specific color, define it in all three blocks (`:root`, the media query, and `[data-theme="dark"]`).

Lightbox (always dark)

The lightbox overlay (the full-screen image/video viewer) has its own dark colors that are theme-independent — the dark background and white controls work in both modes. This is intentional: a dark overlay focuses attention on the content regardless of the page theme.

## Header meta line format

The page header (below the page title) shows a meta line summarizing the directory's contents. The format (Phase 44/45/55) is:




Note the format after the // size segment: the directory count uses a single space separator (not ·), then the rest of the items use ·. This was changed in Phase *55* — the size is visually marked as "special" by the // framing, and the regular meta items after it use a single character (· or space) consistently.

Meta items, in order:

1. Image count — `N images`. Always shown.
2. Video count — `· N videos`. Shown only when `N > 0`.
3. Other files count — `· N other files`. Shown only when `N > 0`.
4. Total size — `// (X.X KB total) //`. Wrapped in // separators (visually distinct from the · separator used for the file counts). Represents the SUM of `Size` for all files in the directory: images + videos + other files. Excludes subdirectories. The literal word `total` follows the size inside the parens, making the meaning clear. Pre-formatted via the `humanSize()` helper (B / KB / MB / GB). After the closing //, the next meta item is separated by a single space (not ·).
5. Directory count — `N directories` (single space separator after //, then the count). Shown when the user is in a subdir (N = subdirs of the current dir) or when there are subdirs in the root listing.
6. Per page size — `· 50 per page`. Always shown.
7. Page indicator — `· Page X of Y`. Shown only when `TotalPages > 1` (multi-page gallery).

Why the size uses // separators instead of ·: the // visually distinguishes the size from the other meta items (which all use ·). The size is conceptually different — it's a quantity (bytes), not a count. The // is a "this is special" marker.

Implementation: the meta line is rendered by the template at the top of the `<style>` block's wrapping `<header>`. The size segment uses three separate `<span>` elements (one for each //, one for the parens) so the browser's flex `gap: 0.5rem` adds visual spacing between them.

## Open-in-new-tab button

The template has a small [] button on each image/video tile (top-right corner of the card) that opens the file's URL in a new browser tab. Added in Phase 47; refined in Phase 52.

Visual: - Position: `top: 6px, right: 6px` inside the `.card` - Size: 28×28 px - Background: `rgba(255, 255, 255, 0.85)` — light translucent - Dark border: `border: 2px solid #000` (Phase 47) so the button stands out more - Arrow color: `#111111` (Phase 52) — a fixed dark color, NOT the theme-aware `--fg` token - The character itself is `north east arrow` (U+2197 NORTH EAST ARROW, displayed as the arrow glyph in the bundled template) - Default opacity: `0.5` (subtle). On hover/focus, opacity goes to `1.0` and the button scales up slightly.

Why the arrow color is fixed `#111111` (not `var(--fg)`): the button has a light translucent background (`rgba(255,255,255,0.85)`) that stays light over ANY page background (light or dark). A dark arrow on a light button background is always visible — no need to adapt to the page's theme. The `--fg` token stays at its current values (`#111111` light / `#e5e5e5` dark) and continues to be used by other elements (h1, body, meta, sort buttons, etc.) — only `.open-btn` is excluded from the theme-aware color rule.

Behavior: - Click or Enter on the button [] `window.open(href, '_blank', 'noopener,noreferrer')` opens the file in a new tab - The `noopener,noreferrer` flags are a security best practice (prevents the new tab from accessing `window.opener`) - The click event is `stopPropagation`'d so it doesn't also trigger the parent's "open in lightbox" handler

## Lightbox controls

The lightbox overlay has 4 buttons + 2 text labels:

| Element | Position | Behavior |
|---|---|---|
| `.lb-close` (×) | Top-right (inside `.lb-controls` pill) | Closes the lightbox; Esc key |
| `.lb-open` ([]) | Top-right (inside `.lb-controls` pill, left of close) | Opens the current image/video in a new tab |
| `.lb-prev` (‹) | Full-height hit area on the left side (120px wide) | Previous image; Left-arrow key |
| `.lb-next` (›) | Full-height hit area on the right side (120px wide) | Next image; Right-arrow key |
| `.lb-counter` | Bottom-left (or bottom-center on mobile) | "N / total" text |
| `.lb-caption` | Bottom-center | The current file's name |

The `.lb-controls` rounded pill (Phase 48): the open ([]) and close (×) buttons are wrapped in a single flex container `.lb-controls` so they appear as a single rounded pill at the top-right of the lightbox. The container has: - Position: `top: 1rem, right: 1.5rem` - Background: `rgba(255, 255, 255, 0.92)` (light pill on the dark lightbox) - Border: `2px solid #000` - Border-radius: `10px`

(the “rounded box”) - Padding: `4px` around the buttons - Display: `flex` (lays out open + close side-by-side, perfectly aligned vertically and horizontally)

The two buttons inside have: - `position: static` (flex lays them out, not absolute) - Transparent background (the container provides it) - No individual border - 28×28 px - Dark icon color (`#1a1a26`) - `border-radius: 6px` on each button (rounded corners within the pill) - Hover: subtle dark background tint (`rgba(0, 0, 0, 0.08)`)

Why a pill instead of separate buttons: a single rounded container gives the two related actions (close + open) a visual unity — they’re both “exit the lightbox” actions (close or view-on-its-own), and grouping them makes that clearer.

The prev/next buttons are NOT in the pill — they’re at the left/right edges of the lightbox. These are navigation actions (different from exit actions), so they get their own positioning.

Prev/next hit areas (Phase 65): the prev/next buttons are full-window-height × 120px wide hit areas positioned at `left: 0` / `right: 0`. The arrow icon is flex-centered inside the hit area. At rest, the hit area is transparent (no visible button) — only a hover reveals the target.

- Hover background — theme-aware:
  - Dark mode (default; the page bg is dark): the hover bg is `rgba(255, 255, 255, 0.08)` — a subtle whiter tint over the dark lightbox. The user sees a soft “highlight” where they’re pointing.
  - Light mode (page bg is light): the hover bg is `rgba(0, 0, 0, 0.06)` — a subtle darker tint. The lightbox itself is still theme-independent (always dark), but the hover tint adapts so it works for visitors on light pages.

Why a full-height hit area: - On touch devices (mobile, tablets), the user doesn’t have a precise cursor — a small button in the middle of the screen is hard to hit. A full-height strip on each side gives a large, easy target. - On desktop, the same hit area means the user can click anywhere in the left or right strip — no need to aim.

Why 120px wide: enough to be a comfortable target on touch screens (~7mm at typical DPI), small enough to leave the center ~60% of the screen clear for the image/video content.

Why transparent at rest: the lightbox is about the content. A visible button on each side would distract from the image. The hover reveals the hit area, so the user sees it when they’re navigating, not when they’re viewing.

JS — `stopPropagation` (Phase 65 fix): the new hit areas sit on top of the media (z-index: 1). The media element has its own click handler that advances to the next image (for `<img>`). Without `stopPropagation` on the prev/next click handlers, a single click in the prev/next area would advance TWICE — once from the button, once from the bubbling media click. The fix: both prev/next handlers call `e.stopPropagation()` before calling `show(idx±1)`.

Why an alpha-blended hover (rgba) instead of a solid color: the lightbox shows whatever media is loaded. A solid bg color would clash with some images (e.g., a green hover on a photo of a red rose). An alpha-blended fill mixes with whatever’s behind, giving a consistent “tint” effect that works on any media.

Why a dark border: like the tile `.open-btn`, the pill has a black 2px border so it stands out clearly against the dark lightbox background. The light pill on a dark bg with a dark border is a strong visual “this is a control” signal.

Keyboard navigation: - `Escape` [] close - `Left arrow` [] previous image - `Right arrow` [] next image - (Clicking on the image itself also goes to next, like carousel UIs)

The `data-theme` attribute on `<html>` is NOT read by the lightbox CSS — the lightbox is intentionally theme-independent (dark always, with light controls). This is the same design choice as the lightbox bg and counter/caption: focus on the content, regardless of page theme.

Video poster (Phase 63): when a video tile has a generated thumbnail (Phase 62's ffmpeg pipeline), the lightbox video element sets its HTML5 `poster` attribute to the thumb URL (extracted from the same `<img class="thumb-img">` element on the tile). The browser shows the poster image immediately when the video opens in the lightbox — the user sees the video's first frame as a still image, then on click the video swaps to playback. This is the same mechanism YouTube uses to show a thumbnail before a video plays.

If `no_video_thumbs` is set OR ffmpeg is missing, the tile has no `<img class="thumb-img">` so the JS can't find a poster URL — the `poster` attribute is simply not set, and the browser shows its default (black frame, or the first decoded frame if `preload="metadata"` is enabled).

The poster URL points at the same cached WebP that's used for the tile thumbnail, so no extra server work or storage is required.

## Pagination

The pagination nav is rendered in two places (Phase 54):

1. Top — after the sort-bar (Name/Type/Modified/Size buttons), before the DIRECTORIES section
2. Bottom — after the IMAGES grid (existing position)

Both pagination navs are identical — same buttons (`Prev`, page numbers, `Next`), same styling, same conditional (only shown when `TotalPages > 1`). Only the position differs.

Why mirror:

- Visitors on a long page don't have to scroll back to the top to switch pages — they can click `Next` at the top.
- The pagination at the top also serves as a "you are here" indicator when arriving on a non-first page (so the user knows they're not on page 1).
- Symmetry: the sort-bar is at the top, the pagination at the bottom; having pagination at the top mirrors the bottom.

The same `{{if gt .TotalPages 1}}` conditional guards both instances, so single-page galleries don't show any pagination at all.

Pagination item format: uses the same Google-style pattern introduced in Phase 29 — `First` (when far from start), `Prev`, page numbers with ellipsis in the middle, `Next`, `Last` (when far from end). The `aria-pressed` is set on the current page number for accessibility.

## Footer

At the bottom of every gallery page, below the IMAGES grid (and below the bottom pagination if multi-page), a small footer credits the underlying technologies (Phases 56/58/59):

```
1 ————————————————————————————————————
2
3            proudly served by caddy + synapticloop // media gallery
```

- "caddy" — links to https://caddyserver.com (the web server)
- "synapticloop // media gallery" — links to https://github.com/synapticloop/caddy_media_gallery (this plugin's repo)

Both links have `rel="noopener" target="_blank"` (security best practice for new-tab links).

Styling: centered, small text (`0.8rem`), muted color (`var(--fg-muted)`), subtle padding. No border-top (removed in Phase 58 — the muted color provides enough visual distinction).

Why these credits: - “caddy” — Caddy is the web server. Without it, this plugin wouldn’t exist; credit the underlying tech. - “synapticloop // media gallery” — links to this plugin’s repo (not just the synapticloop org page) so visitors can read the source, file issues, or fork the project.

The footer text is operator-visible but not configurable — if you want to change it, edit the template (`<footer class="site-footer">` in the bundled template). Operators who fork the project can brand the footer however they like.

## Section heading: “MEDIA” (not “IMAGES”)

The gallery renders both images AND videos (since Phase 25 when the lightbox got video support). To reflect the broader scope, the IMAGES section heading was renamed to MEDIA in Phase 60.

In the bundled template:

```
<h2 class="section-heading">Media</h2>
```

If you fork the template and want to revert to “IMAGES” (e.g., you have a strictly-photo gallery with no videos), change this line. The CSS (`.section-heading`) is unchanged — only the visible text.

The other section headings remain “Directories” and “Other files” (unaffected by Phase 60).

## Mobile: video play button fix (Phase 61)

On mobile devices, clicking the play button on a video in the lightbox used to advance to the next media file instead of starting playback. Root cause: the media click handler always ran `show(idx + 1)` regardless of media type. On mobile, tapping the video’s native play button fires a click event on the `<video>` element, and the handler advanced to the next file BEFORE the video could play. Fix (Phase 61): check `currentEl.tagName === 'VIDEO'` in the handler and bail out so the browser’s native click handling (play/pause) takes over.

For images, click-to-advance still works (unchanged). For videos, click is no longer hijacked — the user can tap the play button on mobile to start playback, or click elsewhere on the video (e.g. the time bar / volume) for the native controls to handle. Navigation is via the prev/next buttons or arrow keys.

## Open-in-new-tab button (Phase 47+52)

The tile-level open-in-new-tab button (the small [] in the top-right of each tile) is documented separately in its own section above. Recent refinements:

- Phase 47 — added the dark border (`border: 2px solid #000`) so the button stands out more. The light translucent bg (`rgba(255,255,255,0.85)`) stays light over both light and dark page bgs, so a dark border is always visible.
- Phase 52 — arrow color fixed at `#111111` (was `var(--fg)`). The user explicitly wanted `--fg` UNCHANGED (still `#111111` light / `#e5e5e5` dark) and used by other elements. The open-btn arrow stays dark in both modes because the button’s bg is always light.

## Building the PDF locally

The PDF book (`caddy-media-gallery-book.pdf`) is built from the markdown files in `docs/` via pandoc + xelatex. The full command is wrapped in a script at the project root: `build-docs.sh`.

To rebuild the PDF locally:

```
./build-docs.sh
```

The script: - Works from any directory (uses `BASH_SOURCE` to find its own location and the project root) - Verifies prerequisites (`pandoc`, `xelatex`) before running - Verifies all source files exist before running - Runs pandoc with the right flags (`--pdf-engine=xelatex`, `--include-in-header=preamble.tex`, `--listings`) - Cleans up intermediate files (`*.aux`, `*.log`, etc.) after - Prints summary info (output path, file size, page count)

Prerequisites: - `pandoc` - `xelatex` (usually in the `texlive-xetex` package) - Install on Debian/Ubuntu: `sudo apt-get install pandoc texlive-xetex texlive-fonts-recommended` - Install on macOS: `brew install pandoc mactex`

Why a script instead of running pandoc directly: the pandoc command has several flags (`--include-in-header=preamble.tex` references the font config; `--listings` is needed for code block formatting; the source file order matters). The script encapsulates all of this so you don't have to remember the incantation.

The font config (`preamble.tex` + `docs/fonts/`) references absolute paths for `fontspec`'s `Path =` option. If you move this project to another machine, update the `Path =` line in `preamble.tex` to match the new location.

# Sort & Pagination

The gallery's image grid respects three URL query parameters. All three are optional. They're a stable URL API — bookmarking a `?sort=size&order=desc&page=3` link works and gives the same view on every visit.

## Query parameters

| Param | Values | Default | What it does |
|---|---|---|---|
| `?sort=` | `name` / `type` / `size` / `mtime` | `mtime` | What field to sort images by. The URL also accepts `?sort=date` (treated as `?sort=mtime`) for back-compat. See "Aliases" below. |
| `?order=` | `asc` / `desc` | `desc` | Sort direction. `desc` for `mtime` is newest-first; `desc` for `name`/`type`/`size` is Z-A / largest-first. |
| `?page=` | integer >= 1 | `1` | Which page of results to show. 1-based. Out-of-range or non-numeric values fall back to 1. |

The URL parameter is preserved across sort changes: clicking "Name" on `?sort=mtime&order=desc&page=2` becomes `?sort=name&order=asc&page=2` (the toggle flips the order for the new field;

the page stays the same).

The directory strip and other-files strip are NOT affected by the sort. They always render in: - Directories: case-insensitive alphabetical (`splitFiles` re-sorts them on every render) - Other files: scanner order (same default as the image list — newest-first by mtime)

This is intentional: the dirs strip is a navigation aid and should be stable, not reshuffle when the user changes the image sort. See Phase 15 in the plan for the reasoning.

## Sort indicator in the header

The header shows the current sort + a reset link:

- On the default sort (`mtime desc`), the indicator is a plain `<span>` — clicking it would just reload the same page, so it's not a link. It reads `Sort: Modified $\downarrow$`.
- On any other sort, the indicator is a link to `?` (no query params, which is the default). It reads e.g. `Sort: Size $\uparrow$`.

## Sort buttons

The sort bar at the top of the header has four buttons: Name, Type, Modified, Size. Each shows a `$\uparrow$` or `$\downarrow$` arrow if it's the currently active sort. Clicking the active button toggles the order; clicking an inactive button switches to that field at the direction that's the "natural" opposite of the current order (so you don't get the same direction twice in a row).

The button labels are produced by the `sortLabel` template function (see the Templates doc for the full map). To add a new sort field:

1. Add a `case` to `sortLabel` in `render.go`
2. Add a `case` to the `sortFiles` function (the sort implementation)
3. Add a button to the `gallery.tmpl` template

It's a code change in three places; the template is just the visible button.

## Pagination

50 images per page (constant `PageSize = 50` in render.go, not configurable via env var or Caddyfile — it's a code constant).

Pagination links in the live template: - The "First" and "Prev" pagination buttons (rendered by the template with double- and single-left-arrow icons, when `{{.HasPrev}}` is true) - The "Next" and "Last" pagination buttons (single- and double-right-arrow icons, rendered when `{{.HasNext}}` is true) - Current page indicator in the middle

The page numbers in the URL are 1-based, not 0-based. `?page=0` falls back to `?page=1`.

## Examples

```
/images/                          # default: mtime desc, page 1
/images/?sort=name&order=asc      # alphabetical
/images/?sort=size&order=desc     # largest first
/images/?page=3                   # third page of default sort
/images/?sort=type&order=asc&page=2
```

### Aliases

`?sort=date` is an alias for `?sort=mtime` — both sort by the file's modification time. The two values produce identical results:

```
/images/?sort=date&order=desc    # == /images/?sort=mtime&order=desc
/images/?sort=date&order=asc     # == /images/?sort=mtime&order=asc
```

Why have the alias? The two names describe the same thing from different angles — "mtime" is the technical field name (modification time, from the filesystem), "date" is the user-friendly name (the date the file was last changed). The "Modified" sort button in the UI emits `?sort=mtime` (the technical name), but the URL API accepts both for back-compat with bookmarks and external links that might use either name.

Implementation: `parseSort` in `render.go` accepts `"date"` in its switch statement and treats it identically to `"mtime"`:

```
switch field {
case "name", "type", "date", "mtime", "size":
    // all valid; "date" is an alias for "mtime"
default:
    field = "mtime" // unknown fields fall back to the default
}
```

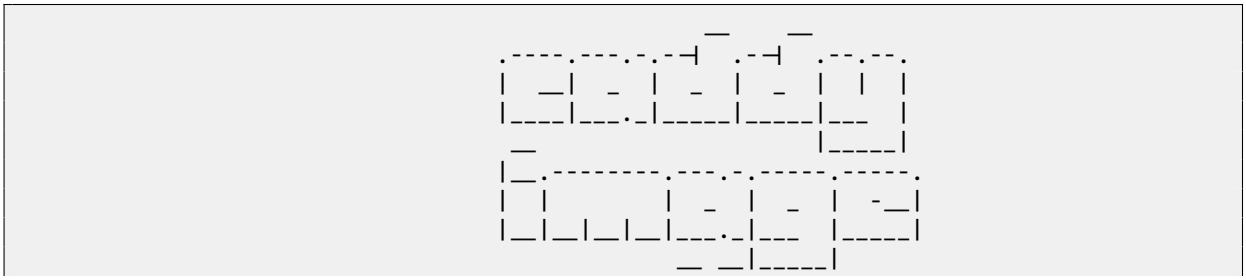
In `sortFiles`, both `"mtime"`, `"date"`, and `""` (the empty default) share the same sort branch — sort by `ModTime` per the `order` param. So functionally there's no distinction between them.

When to use which: if you're writing a new integration or bookmark, prefer `?sort=mtime` (it's the canonical name and what the UI button emits). `?sort=date` is fine too — just pick one and stick with it for consistency.

### Stability guarantees

- The URL parameter API is stable. Bookmarking works. The four sort fields and two orders are part of the public contract; renaming or removing one is a breaking change.
- Adding a new sort field is non-breaking (just adds a new option).
- The default sort may change in a future major version, but the `?sort=mtime` URL will always be honored.
- Page size is a code constant and may change in future versions. URLs with `?page=N` are stable relative to the *current* page size — if the page size changes from 50 to 100, page 3 of the old URL might now be page 2 of the new layout, but no image is lost or duplicated (it's the same image set, just packed differently).

## 05. End Plate



```
                                  .-----.---.-|  |  .-----.----.--.--.
                                  |  _  |  _  |  |  |  -__|   _|  |  |
                                  |___  |___._|__|__|_____|__| |___  |
                                  |_____|                     |_____|


                                             "Parting is such sweet sorrow"

                                                                                 Romeo And
    Juliet
                                                                                  Act 2,
    Scene 2,
    -176185
```